

How ParkProof was built

The explanation — for me, in plain English.

An ELI15 walk through the stack, the decisions, and the why.



Contents

1. The big picture
2. The frontend - what runs in your browser
[PARKPROOF · INTERNAL](#)
3. The backend - one Lambda function pretending to be 10 servers
4. The AI brain - how Claude actually reads the sign
5. Where the data lives
6. Cloud sync, auth, and the bouncer
7. Cryptographic evidence sharing
8. Hosting the website
9. The polish - optimisations that make it feel finished
The explanation — for me, in plain English.
10. The feedback loop - how I would know if it is working
An ELI15 walk through the stack, the decisions, and the why.
11. What it costs to run

Use the PDF sidebar bookmarks for one-tap jumps to any section.



1. The big picture

ParkProof is a website that pretends to be an app. You open it on your phone, you can **install it to the home screen with its own icon**, and from then on it behaves like a native app — full-screen, no browser address bar, offline-aware. The technical name for this is a **PWA**: a Progressive Web App.

Why a PWA and not a real iOS/Android app? Two reasons. First, you ship one codebase instead of three (iOS, Android, web), which matters a lot when you're solo. Second, you skip the Apple and Google app stores entirely — no \$99/yr Apple developer fee, no two-week App Store review, no risk of being rejected for breaking some rule about parking signs. The downside is that a few things native apps get for free (background notifications, deep camera APIs) are harder or impossible in a PWA. ParkProof's whole feature set is chosen to live inside what a PWA can do.

The product is **mobile-first** because the user is literally standing next to a parking sign with one hand free. The entire home screen is one big blue button: *Scan a parking sign*. Everything else — history, settings, language picker — is one tap deeper.

What you experience: tap the button, the camera opens, you photograph the sign, an AI reads it, and ~8–12 seconds later you see a green or red card that says *You can park here · 1h 58m left* or *You cannot park here*. From there you can save the moment with a GPS-tagged car photo, set a calendar reminder, and later export the whole thing as a PDF you'd hand to a council in a dispute.

Key takeaway — *ParkProof is a PWA because the alternative was building the same app three times and paying Apple to review each one. The mobile-first design is because the user is on a footpath, not a desk.*

2. The frontend - what runs in your browser

The visible part of ParkProof — every button, every screen, every animation — is written in **React** with **TypeScript**, styled with **Tailwind CSS v4**, and bundled by **Vite**. Four words; let's unpack each.

React

React is the library that lets you describe what the screen *should* look like given the current state of the app, and then it figures out the minimum number of DOM updates needed to get from where the screen is now to where you said it should be. Without React, you'd be writing “find element X, change its text, also hide element Y, also...” by hand for every interaction. React lets you write “here's what this screen looks like when there are 5 saved sessions” and lets the library handle the diffing.

TypeScript

TypeScript is JavaScript with a type system bolted on. You declare what shape your data is (*a ParkingSession has an arrived_at, a location, a sign_photo...*) and the compiler catches mismatches before they reach the browser. The cost: a build step and some extra typing. The benefit: roughly half of the bugs you would have shipped never exist.

Why TypeScript over plain JavaScript? Plain JS lets you write `session.localiton.address` (typo!) and find out about it at 2am when a user reports the address is missing. TypeScript flags it the moment you type it. On a solo project where there's no one else to catch your mistakes, this is the single highest-leverage tool you can use.

Tailwind CSS v4

Tailwind is a CSS framework where instead of writing a stylesheet that says `.button { padding: 12px; background: blue; }`, you write the styles inline on the element as classes: `<button className="p-3 bg-brand-500">`. It feels backwards the first time you see it, but the benefits are real: no naming things (which is famously hard), no jumping between two files to find a style, and the final CSS bundle only contains the classes you actually used.

Why Tailwind over CSS files? Speed of iteration mostly. When the design is evolving — and a portfolio project's design evolves a lot — being able to change padding without renaming a CSS class or worrying about specificity is genuinely faster. The downside is the JSX gets visually noisy. I'd rather noisy and fast than tidy and slow.

Vite

Vite is the build tool — it turns the TypeScript + React + Tailwind source code into the JavaScript bundle the browser actually downloads. It's also the dev server that gives you instant hot-reload while you're working (save a file, see the change in your browser within ~50ms). It replaced the older Webpack/Create-React-App ecosystem in the early 2020s by being roughly 10× faster.

Key takeaway — *Frontend in one sentence: React describes the UI, TypeScript catches the bugs, Tailwind styles it fast, Vite builds and serves it.*

3. The backend - one Lambda function pretending to be 10 servers

Every time you tap “Scan a parking sign”, your phone sends a request to a server. Where does that server live, and what's running on it?

ParkProof doesn't have a server in the traditional sense. There's no machine always running waiting for a request. Instead it uses **AWS Lambda**: when a request comes in, AWS spins up a tiny computing environment, runs your code, returns the response, and shuts it back down. You pay for milliseconds of compute used, not hours of a server being on. This is called **serverless** — a slightly misleading name (there ARE servers, you just don't manage them).

Why one function for ten routes?

ParkProof has ten API routes — /sign-translate, /draft-appeal, /sign-session, /feedback, plus six more for the cloud-sync and account features. The natural instinct is to make each route its own Lambda. I made one Lambda handle all ten by dispatching on the URL path.

Why? **Cold starts**. Lambdas that haven't been called recently take ~1 second to start the first time. If you have ten Lambdas, you have ten potential cold starts. One Lambda means once it's warm, every route is fast. At ParkProof's traffic (~100 invocations a month), this is the difference between “the first user every hour waits a second” and “most users get an instant response”. At Google's traffic, you'd split them up. At ParkProof's traffic, one function is the right call.

API Gateway - the front door

Browsers can't call Lambda directly. **API Gateway** sits in front of Lambda and gives you a stable URL ([https://t1smpbft4f.execute-api.ap-southeast-2.amazonaws.com/...](https://t1smpbft4f.execute-api.ap-southeast-2.amazonaws.com/)) that handles HTTPS, CORS, request validation, rate limiting, and authentication. Think of it as the building's front desk: it accepts deliveries, checks IDs for the floors that require them, and forwards everything to the right office inside.

Six of the ten routes (the cloud-sync ones) are gated by a **JWT authorizer** configured at the API Gateway level. That means API Gateway checks the user's token *before* the request ever reaches Lambda. If your token is missing or expired, you get a 401 and we don't burn compute on you. The other four routes (sign-translate, draft-appeal, sign-session, feedback) are anonymous — anyone can call them.

Key takeaway — One Lambda handling all the routes is right for this scale. It would be wrong for a high-traffic product (you'd want isolation), and it would be wrong for a multi-team org (you'd want ownership boundaries). At one developer, one user, one domain — it's perfect.

4. The AI brain - how Claude actually reads the sign

When you photograph a parking sign, the Lambda function takes that image, packs it into a request, and sends it to Anthropic's API. The model on the other end is **Claude Sonnet 4.6**. A few things make this work.

Vision

Sonnet 4.6 is a **multimodal** model — it can take images as input, not just text. We send the image as a base64-encoded string alongside the system prompt (a long set of instructions about how to read Australian parking signs) and a small bit of context (the current time, the user's GPS coords resolved to a timezone). The model returns a structured JSON response with the rules it saw, whether you can park right now, and when you'd have to leave by.

Adaptive thinking

Parking signs in Melbourne CBD are nasty. A typical inner-city pole has three signs stacked: maybe *2P 8am–6pm Mon–Fri* on top, *1/4P 8am–6pm Mon–Fri* in the middle, *Permit Zone 8am–11pm Sat–Sun* at the bottom. The model has to compute the “leave-by” time for *each* rule, then pick the earliest one — because that's the one that catches you first.

This is multi-step reasoning, and it goes wrong without dedicated “thinking” time. Sonnet 4.6 supports **adaptive thinking** — the model decides how much internal reasoning to do before answering, and that thinking happens in its own private scratchpad before the final response. We tell it thinking: `{ type: 'adaptive' }` and let it pick. The result is correct math on stacked signs.

I tried the cheaper, faster Haiku 4.5 model. It doesn't support adaptive thinking at all. Without it, Haiku consistently picked the *latest* leave-by time instead of the earliest — exactly backwards. Sonnet 4.6 is the floor for this task; cheaper is not an option.

JSON schema enforcement

Asking an AI for “structured output” used to be a roll of the dice — you'd write a prompt saying “return JSON like this” and the model would mostly comply, occasionally returning markdown or a bonus sentence of explanation that broke your parser. Sonnet 4.6 lets you specify the exact JSON shape as a schema, and the model is *guaranteed* to return JSON matching it. No more parsing failures, no more “I hope it remembered to escape the quotes”.

Smart re-scan - text-only mode

Here's the optimisation I'm proudest of. If you parked at the same spot 6 hours ago and you're back now, ParkProof doesn't need to re-read the sign — the rules haven't changed. The signage is still the same physical object. What's changed is the time.

So instead of sending the photo again, the Lambda sends Claude only the rules text from the prior reading, plus the current time, and asks: “Given these rules, can the user park right now, and until when?” This is text-only, shorter, and skips the vision pipeline entirely. The result: **~3× faster and ~4× cheaper** per re-scan.

The trigger is automatic: when you tap “Scan a parking sign” near a spot you've scanned in the last 7 days (within 40 metres), you see a “You scanned this spot before — reuse the reading?” card. Tap it, and the refresh runs.

Key takeaway — *The AI brain is Sonnet 4.6 with adaptive thinking and JSON-schema output. Don't downgrade. Don't change the schema. Don't remove the worked examples in the system prompt — they were added after a regression where the model picked the wrong leave-by time.*

5. Where the data lives

ParkProof stores user data in three different places, picked for what kind of data it is and how durable it needs to be.

localStorage - on your phone

Every browser has a small key/value storage that survives page reloads but never leaves the device. It holds ~5MB per origin. ParkProof writes all your saved sessions to localStorage *first*, before anything else happens. That means everything works offline: scan a sign with no signal, save the session, set a reminder — all of it works on local storage alone.

The cost of localStorage is the 5MB ceiling. Photos eat that fast. ParkProof resizes every photo down to 1200px on the longest edge and re-encodes as 82%-quality JPEG before it touches storage — turning a 5MB raw phone photo into a ~200KB stored one. There's also a 3-phase auto-recovery: if storage hits the ceiling, the app strips car photos from expired sessions first, then sign photos, then evicts whole expired sessions. Active sessions are never touched.

DynamoDB - the cabinet in AWS

If you sign in (optional), the same sessions get mirrored to **DynamoDB**, AWS's managed NoSQL database. Think of it as a giant filing cabinet that AWS operates for you — you don't run the database server, you just send “store this” and “read all rows where userId = X” requests to its API.

ParkProof uses one table, parkproof-sessions, with two keys: userId as the partition key (which user) and sessionId as the sort key (which session). That structure means listing one user's sessions is a single, fast query — DynamoDB knows exactly which physical disk to look at.

S3 - the warehouse for photos

Photos don't go into DynamoDB (it's expensive to store binary blobs there). They go into **S3**, AWS's object storage — basically a giant warehouse where you can stash files of any size, organised by “buckets” (top-level containers) and “keys” (paths inside the bucket).

ParkProof has two buckets: parkproof-app-... for the static website files (what CloudFront serves to your browser) and parkproof-evidence-... for user-uploaded photos. The evidence bucket organises photos under per-user prefixes ({userId}/sessions/{sessionId}/sign.jpg) so each user's files are cleanly partitioned.

Why local-first?

The natural SaaS instinct is “require sign-up so you can build a user funnel”. I deliberately did the opposite. A user who's just been screwed by a parking ticket is in *no mood* to create an account, verify an email, and agree to a privacy policy before they can translate a sign. Anonymous, local-first means there's zero friction tax on first use. The cloud sync (DynamoDB + S3) is a bonus you opt into when you want cross-device durability.

Key takeaway — *Local-first is the default; cloud is the opt-in. The local copy is the source of truth — the cloud is just durability insurance.*

6. Cloud sync, auth, and the bouncer

When you opt into cloud sync, ParkProof needs to know who you are without trusting your browser to just declare it. That's what **Amazon Cognito** does.

Cognito - the identity service

Cognito is AWS's managed user identity service. It handles the boring, dangerous stuff: password hashing, email verification, password reset, login throttling. ParkProof has one **User Pool** (a directory of users) plus a **Hosted UI** domain that handles the OAuth dance with Apple and Google when users choose “Continue with Apple” or “Continue with Google” instead of creating an email/password account.

JWT - the wristband

When you sign in, Cognito gives your browser three tokens: an **idToken**, an **accessToken**, and a **refreshToken**. The first two are **JWTs** — JSON Web Tokens — which are essentially signed JSON blobs that say “this user is who they claim to be, here's their `userId`, this token is valid for the next hour”. Cognito signs them with a private key only it has, so nobody can forge one.

Think of a JWT as a festival wristband. You got it at the entrance after showing your ID. Every time you want to go into a specific tent, the staff at the door just looks at the wristband — they don't need to call dispatch to re-verify you. The cryptographic signature is what makes the wristband impossible to fake.

The bouncer - API Gateway authorizer

Six of ParkProof's ten API routes are auth-gated. On each request to those routes, your browser attaches the JWT as an `Authorization: Bearer ...` header. **API Gateway's JWT authorizer** intercepts the request before it reaches Lambda, verifies the signature against Cognito's public key, checks the expiry, and (if valid) puts the user's `sub` claim (their unique ID) onto the request as metadata. Lambda then reads that `sub` and only returns data belonging to that user.

If the token is bad — missing, expired, signature mismatch — API Gateway returns a 401 directly without ever invoking Lambda. You don't pay compute time for rejecting unauthorised requests.

Federation - let people use existing IDs

Most users don't want yet another email/password to remember. Cognito Hosted UI supports OAuth federation with Apple and Google, which means “Continue with Apple” on the ParkProof sign-in screen redirects you to Apple's login page, you authenticate there (Face ID on iPhone, usually), and Apple sends you back to ParkProof with a token Cognito accepts. ParkProof never sees your Apple password and Apple never sees your ParkProof activity — Cognito is the trusted broker.

Key takeaway — *Cognito holds identities, API Gateway checks tokens, Lambda only ever sees verified user IDs. Apple and Google federation means most users never have to create yet another password.*

7. Cryptographic evidence signing

Here's the bit of ParkProof that gets the most “you did *what?*” reactions when I explain it. Every saved session is signed by an AWS-held key, so years from now someone can verify that the evidence record hasn't been tampered with.

Why signing, not just hashing?

If you just **hash** the photo (compute SHA-256), you can later show the same photo produces the same hash — but the user could have altered the photo right after saving and re-hashed it. The hash on its own proves nothing about *when* the photo existed in that form.

A **signature** is different. It's a hash that's been encrypted with a private key the user does not have access to. Anyone can verify the signature using the matching public key (which we publish openly), but only the holder of the private key could have produced it. So if you can verify a signature, you know: (a) the data hasn't changed since it was signed, AND (b) someone with access to the private key produced this signature at some point. The signed metadata includes a timestamp, so “at some point” becomes “at this specific moment”.

KMS - the notary

AWS **KMS** (Key Management Service) holds the private key. The key never leaves AWS — there's no API to download it, ever. The only thing you can ask KMS to do is “please sign this blob with the private key and return me the signature”. It's like having a notary public who'll stamp your documents but won't lend you their stamp.

ParkProof's key is **ECDSA P-256** (Elliptic Curve Digital Signature Algorithm, 256-bit curve). It's an asymmetric algorithm — there are two keys, one for signing (private, held by KMS) and one for verifying (public, shipped at /parkproof-public-key.pem on the website). Anyone can download the public key and verify any signature ever produced by the private key.

The PDF appendix

When you export a session as an evidence PDF, the last page is an “Evidence verification” appendix. It includes the raw signature bytes (base64), the canonical data that was signed (timestamps, GPS, photo SHA-256 hashes), and a one-paragraph walkthrough showing exactly how to verify it with `openssl dgst -verify` against the public key. A council, court, or insurer can verify the evidence chain without ParkProof being in the loop. *That's* the real differentiator: not the AI, not the GPS — the verifiability.

Key takeaway — A hash proves “these bytes match”. A signature proves “these bytes existed in this exact form at this specific moment, witnessed by someone with the private key”. ParkProof signs everything important.

8. Hosting the website

How does `www.parkproof.com.au` serve files to a phone in Melbourne (or anywhere else) fast and cheaply? Four moving parts.

S3 - the warehouse

The static website (HTML, JS bundles, CSS, icons, the parking-sign images, the OG card) lives in an S3 bucket called `parkproof-app-....` S3 is cheap, durable, and infinitely scalable, but it's not optimised for low-latency delivery — a fetch from S3 in Sydney to a phone in Perth is fine, but to a phone in London it'd be sluggish.

CloudFront - the worldwide delivery network

CloudFront is AWS's CDN — Content Delivery Network. It has ~400 edge locations around the world, each of which caches a copy of your website's files. When someone in London visits the site, CloudFront serves them from its London edge — no round-trip to Sydney needed. When someone in Melbourne visits, they get served from MEL51-P2, an edge POP about 5km from the CBD.

Edge locations only pull from S3 on cache miss. After the first request to each asset, subsequent requests in that region are served from the edge cache. That means S3 sees maybe a few hundred requests per month even at moderate traffic. It also means an invalidation step at deploy time — “forget the old version, fetch the new one”.

OAC - the rule “only CloudFront can pull”

The S3 bucket is *private* — direct requests to it are rejected. The only thing allowed to pull from it is CloudFront, via a configuration called **Origin Access Control** (OAC). This is a security best-practice: it stops anyone from bypassing CloudFront and hitting S3 directly (which would skip the edge caching and could rack up egress bills if abused).

ACM + DNS - the custom domain

By default, CloudFront gives you a URL like `d1jmpu2roekssu.cloudfront.net`. Functional, ugly. To use `www.parkproof.com.au` instead, you need two things: an SSL certificate for that domain (free from AWS Certificate Manager, **ACM**) and a DNS CNAME record at Cloudflare pointing `www.parkproof.com.au` → `d1jmpu2roekssu.cloudfront.net`. The apex `parkproof.com.au` (and both forms of `parkproof.au`) 301-redirect to the canonical via Cloudflare Page Rules.

(One memorable diversion: Network Solutions, where the domain is registered, has a wildcard A record on all subdomains pointing to their “under construction” parking page. Even after adding the CNAME, browsers kept hitting the parking page because the wildcard A was beating the explicit CNAME in their DNS responses. Fixed by deleting the wildcard. Total time

to debug: longer than the actual implementation.)

Key takeaway — S3 stores it, CloudFront delivers it worldwide, OAC keeps S3 private, ACM + DNS give it a real domain name. The whole hosting stack runs at well under \$1/month.

9. The polish - optimisations that make it feel finished

A handful of small things, each of which would be invisible if it weren't there but together account for the “this feels like a real app” impression.

Photo resize

Every photo gets canvas-resized to 1200px on the longest edge and re-encoded as 82%-quality JPEG before it ever hits storage or the API. A 5MB phone photo becomes ~200KB. The visible quality difference is none; the storage and API-payload savings are 25×.

localStorage quota recovery

If localStorage hits its 5MB ceiling, a 3-phase recovery runs against expired sessions only:

Phase 1 strip the car photos, **Phase 2** strip the sign photos, **Phase 3** evict the whole session record. Active sessions (and anything signed) are never touched. Without this, a heavy user would hit the QuotaExceededError within 1–2 sessions and the save flow would silently break.

i18n - 7 languages

The whole UI is internationalised via react-i18next, with 7 language packs: English, Mandarin, Vietnamese, Italian, Greek, Hindi, Punjabi. The list wasn't arbitrary — it's the top non-English languages spoken at home in the City of Melbourne LGA per the 2021 Census. The model output (the rules text) stays in English because that's what's literally on the sign; everything around it translates.

PWA install

Two files do most of the PWA work: a manifest.webmanifest (tells the browser the app's name, icon, theme colour, and start URL when installed) and a service-worker.js (a tiny script that runs even when the tab is closed, handling offline caching). The Vite PWA plugin generates both from configuration. The Apple touch icon and various Android icon sizes are auto-generated from a single source SVG.

Code-splitting

Heavy libraries (jsPDF for evidence export, ics for calendar generation, html2canvas for the PDF photo overlay, DOMPurify for sanitising AI letter output) are **lazy-loaded** — they only download when the user actually triggers the relevant feature. Result: the main bundle stays at ~225KB gzipped, fast on a 3G connection. Without code-splitting, the bundle would be 600KB+ and the first paint would noticeably stutter.

Key takeaway — *Each of these is small in isolation. Together they're the difference between “a developer wrote this” and “a product team shipped this”.*

10. The feedback loop - how I'd know if it's working

This is the part of the build I'm most proud of from a PM perspective, even though no real user has ever fired it. After each AI translation, the user sees two buttons: “Yes, looks right” and “Retake photo”. Tapping either sends a small JSON event to AWS CloudWatch Logs.

Layer 1 - verdict counts

The simplest version. Each event includes a verdict (correct or retake) and a UUID. Aggregated across all users, this gives you a single number: *the percentage of translations users actually trusted*. If it's 95%, the model is generally right. If it drops to 70%, something's broken.

Layer 2 - verdict + context

Layer 1 tells you *if* there's a problem. Layer 2 tells you *what kind*. Each event now also carries: the model's stated confidence (low/medium/high), whether the clarification step fired (i.e. was this a multi-variant sign?), the local hour of day, a 120-character excerpt of the rules text, and a flag for whether it was a smart re-scan (text-only refresh) or a fresh vision call.

With Layer 2 in place, you can slice the failure modes in **CloudWatch Logs Insights** (AWS's query tool for log data) and ask questions like:

- “Of all retake verdicts, what's the model's confidence distribution?” If retakes correlate with low confidence, the model is well-calibrated. If they correlate with *high* confidence, that's a prompt regression and I need to look immediately.
- “Does retake rate spike at certain hours of day?” A retake spike at 8pm+ is a photo-quality issue (dim light), not a model issue.
- “Are multi-variant signs (*clarification = true*) more error-prone than single signs?” If yes, that's where prompt work should focus.

Why this is the part most projects skip

Most AI features ship without a feedback loop at all. The team ships the model, the model degrades silently as the data drifts, and nobody knows until users start churning. Building the telemetry *before* there are users is exactly the wrong-feeling order — it's instinct to wait until you have real traffic. But the moment you have real traffic, you also have urgency, and the telemetry never gets built. Layer 1 + Layer 2 are five days of work that pay back tenfold the first time a regression silently lands.

Key takeaway — *Telemetry is the part where the discipline of building the unsexy thing first compounds. Layer 1 alone is what most projects skip. Layer 2 is what separates “we built an AI feature” from “we operate an AI feature”.*

11. What it costs to run

At portfolio traffic (~100 visits per month, ~30 sign-translate calls, ~5 appeal drafts), ParkProof costs ~\$5–7 per month. Where does that go?

The fixed costs (~\$4–5/month)

- **KMS asymmetric key — \$1.00/month.** AWS charges this whether the key is used or not. It's the fee for keeping a customer-managed cryptographic key in their vault. The signing operations themselves are essentially free at our volume.
- **Domain renewal — \$3–4/month amortised.** .tech domains at Network Solutions renew ~\$40–50/year. The only non-AWS fixed cost.

The usage costs (~\$1–3/month at portfolio traffic)

- **Anthropic API — \$1.50–3/month.** About \$0.05 per fresh sign translate (image-bearing call to Claude Sonnet 4.6), about \$0.01 per refresh (text-only smart re-scan), about \$0.06 per appeal draft. The image-bearing calls dominate.

Effectively free under AWS tiers

- Lambda — well under the 1 million-requests/month always-free tier.
- API Gateway HTTP — pennies at this volume even past the 12-month free tier.
- CloudFront — under the 1TB/month free transfer cap.
- S3 — both buckets store <10MB, well under free-tier thresholds.
- DynamoDB — on-demand at our volume is effectively free.
- Cognito — 50,000 monthly active users free forever.
- CloudWatch Logs — under 1MB ingested per month.
- ACM cert — free.

The Budgets alarm

An **AWS Budgets alarm** is set at \$10/month, emailing me if total spend trends to exceed that. It's the safety net for the “a tech blog picked it up and 10,000 strangers are now scanning signs” scenario, where the Anthropic costs would balloon to ~\$500 in a day. The AWS surface itself scales gracefully — even at 1000× traffic, AWS stays under \$20/month. Anthropic is the uncapped variable.

Key takeaway — Two costs dominate: KMS (\$1) and the domain (~\$4 amortised). Everything else is effectively free at portfolio scale. The \$10/month budget alarm exists to catch the one scenario where a viral moment turns into a \$500 Anthropic bill.

That's the whole stack. Eleven layers, each picked for a specific reason, each documented somewhere in the repo so you can re-read it in a year and remember why it's there. The discipline of writing it down is the same discipline that made the build possible: do the unsexy thing first, defer explicitly, and let the trade-offs be visible.